

Simulating Logistics Systems

Lesson 5/15

João Flávio de Freitas Almeida

LEPOINT: Laboratório de Estudos em Planejamento de Operações Integradas
Departamento de Engenharia de Produção

Universidade Federal de Minas Gerais
Escola de Engenharia

UFMG, Belo Horizonte, Brasil
10.09.2025

Outline of this lecture

Simpy Models

SimPy Exercise

Simpy Models

SimPy Exercise

Outline of this lecture

Simpy Models

SimPy Exercise

Simpy Models

SimPy Exercise

(Python) generator: `yield` [SimPy Contributors, 2025]

SimPy is a **process**-based **discrete-event simulation** framework based on **Python** [SimPy Contributors, 2025, Medina, 2025]

- ▶ **Generator**: A (python) function which returns a generator iterator. It looks like a normal function except that it contains **yield** expressions for producing a series of values usable in a for-loop or that can be retrieved one at a time with the `next()` function. Each `yield` temporarily suspends processing, remembering the execution state. When the generator resumes, it picks up where it left off (in contrast to functions which start fresh on every invocation) [Python Contributors, 2025].

Python Generator Example

```
def simple_generator():  
    yield 1  
    yield 2  
    yield 3  
  
# Create generator object  
gen = simple_generator()  
  
# Use next() to get values one by one  
print(next(gen))  # 1  
print(next(gen))  # 2  
print(next(gen))  # 3  
  
# If we call next() again, it raises StopIteration  
# print(next(gen))  # Uncommenting will raise StopIteration
```

Python Generator

```
# Generator function that counts down from n to 1
def countdown(n):
    print("Counting down from", n)
    while n > 0:
        yield n
        n -= 1
    print("Done!")

for i in countdown(5):
    print(f"Generator yield: n={i}")
```

First Steps in SimPy

- ▶ Importing libraries: SimPy, random
- ▶ Creating simulation environment
- ▶ Creating arrivals within simulation
- ▶ Creating time intervals
- ▶ Running the simulation model

First Steps in SimPy: Importing libraries

simpy is SimPy itself, and **random** is a random number generation library.

```
import simpy
import random

random.seed(1)
```

Using `random.seed(1)` ensures that the sequence of random numbers generated each time the program is run is always the same, facilitating verification.

First Steps in SimPy: Creating simulation environment

```
import simpy
import random

random.seed(1)
env = simpy.Environment()
```

In SimPy, everything revolves around events generated by functions. These events must occur within a simulation "environment" created using the `simpy.Environment()` function.

First Steps in SimPy: Creating arrivals

```
import simpy
import random
random.seed(1)

def generateArrivals(env, name, rate):
    pass

env = simpy.Environment()
env.process(generateArrivals(env, "Patient", 2))
```

We write a `generateArrivals()` function that creates entities in the system for the simulation. Our entity generator has three input parameters: the **environment**, an attribute to represent the **name** of the entity, and the **rate** of entity arrivals per unit of time. In SimPy, you will build an event generator function within the created environment. The events generated will be the arrivals of entities in the system.

First Steps in SimPy: Creating time intervals

```
def generateArrivals(env, name, rate):
    countArrivals = 0
    while True:
        yield env.timeout(1/rate)
        countArrivals += 1
        print(f"{env.now}: {name} {countArrivals} arrived!")

env = simpy.Environment()
env.process(generateArrivals(env, "Patient", 2))
```

The **while** loop repeats forever during the simulation. The counter, **countArrivals**, stores the total number of entities generated, and the print function displays the arrival time of each patient. The print function uses the current simulation time **env.now**.

First Steps in SimPy: Running the model

```
import simpy
import random
random.seed(1)

def generateArrivals(env, name, rate):
    countArrivals = 0
    while True:
        yield env.timeout(1/rate)
        countArrivals += 1
        print(f"{env.now}: {name} {countArrivals} arrived!")

env = simpy.Environment()
env.process(generateArrivals(env, "Patient", 2))
env.run(until=11)
```

Simpy: Process

```
# The SimPy process uses a yield function. These processes
    produce (yield) events that are sequenced by the environment
    (env). A "process" can be paused (yield) and resumed later
    ..

import simpy

def ex_process(env):
    print(f'{env.now}: Inicio do processo')
    yield env.timeout(5)  # 'Advance' 5 time units
    print(f'{env.now}: Proccess finished')

env = simpy.Environment()

env.process(ex_process(env))
env.run(until=10)  # Run simulation for 10 time units
```

Simpy: Events

```
# Events are specific points in the simulation where actions
    occur, like a doctor finishing a consultation or a bed being
    requested. Timeouts are the most basic event in Simpy. Use
    env.timeout(t) to simulate the time evolving.

import simpy

def ex_event(env):
    print(f'{env.now}: Atendimento iniciado')
    yield env.timeout(20) # The service is provided for 20
        units of time.
    print(f'{env.now}: Service finished')

env = simpy.Environment()
env.process(ex_event(env))
env.run(until=50) # Run simulation for 50 time units
```

Simpy: A simple program (function)

```
import simpy

# 1. Create a simpy environment
# 2. Define a process that will run in the environment
# 3. Add the process to the environment
# 4. Run the simulation for 100 time units

def ambulancia(env):
    while True:
        print(f'{env.now}: Ambulance arrived at hospital.')
        yield env.timeout(5) # Ambulance at hospital por 5
        print(f'{env.now}: Ambulance search patients.')
        yield env.timeout(30) # Ambulance search patients.

env = simpy.Environment() # 1. Create simpy environment
env.process(ambulancia(env)) # 3. Process at env
env.run(until=100) # 4. Run simulation
```

Simpy: Resource

```
import simpy

def paciente(env, name, tempo_de_atendimento):
    print(f'{env.now}: {name} arrive at hospital')
    with atendente_triagem.request() as req:
        yield req
        print(f'{env.now}: {name} begins service.')
        yield env.timeout(tempo_de_atendimento)
        print(f'{env.now}: {name} finish service.')

env = simpy.Environment()
atendente_triagem = simpy.Resource(env, capacity=1)
env.process(paciente(env, 'Paciente 1', 5))
env.process(paciente(env, 'Paciente 2', 3))
env.run(until=20)
```


Outline of this lecture

Simpy Models

SimPy Exercise

Simpy Models

SimPy Exercise

SimPy Exercise: Implement your project model

Exercise: The group should retrieve the simulation project model. Using SimPy features listed below, implement the logic of the ACD proposed in the last class.

- ▶ Process
- ▶ Events
- ▶ Resource
- ▶ Functions

The model should **print events over time**. *Implementing data analysis and system performance indicator features is not necessary at this point.*

Outline of this lecture

Simpy Models

SimPy Exercise

Simpy Models

SimPy Exercise

```
TAXACHEGADAS = 2

def geraChegadas(env, nome, TAXACHEGADAS):
    contaChegada = 0
    while True:
        # Chegadas com intervalos de 0.5 min.
        yield env.timeout(random.expovariate(1.0/TAXACHEGADAS))
        contaChegada += 1
        print("%s %i chega em: %.1f" % (nome, contaChegada, env
            .now))

env = simpy.Environment()
env.process(geraChegadas(env, "cliente", TAXACHEGADAS))
env.run(until = 10)
```

```
def geraChegadas(env, nome, taxa, numeroMaxChegadas):
    contaChegada = 0
    while (contaChegada < numeroMaxChegadas):
        yield env.timeout(random.expovariate(1.0/taxa))
        contaChegada += 1
        print("%s %i chega em em: %.1f" % (nome, contaChegada,
            env.now))

env = simpy.Environment()
env.process(geraChegadas(env, "cliente", 2, 12))
env.run()
```

SimPy: Arrivals [SimPy Contributors, 2025]

```
(venv) PS C:\Users\user\Desktop\s1s\07> py .\ex1-Intro-Funcao-Chegadas.py  
3.13.3 (tags/v3.13.3:6280bb5, Apr 8 2025, 14:47:33) [MSC v.1943 64 bit (AMD64)]  
cliente 1 chega em: 1.6  
cliente 2 chega em: 2.7  
cliente 3 chega em: 6.4  
cliente 4 chega em: 6.6  
cliente 5 chega em: 8.8
```

SimPy: Dict of Distributions [SimPy Contributors, 2025]

```
def distribuicoes(tipo):  
    return {  
        'chegada': random.expovariate(1/1.0),  
        'cantando': random.triangular(10,30,20),  
        'aplauso' : random.gauss(10,1),  
    }.get(tipo, 0.0)  
  
def geraChegadas(env, nome, numeroMaxChegadas):  
    while (contaChegada < numeroMaxChegadas):  
        yield env.timeout(distribuicoes('chegada'))  
        print("%s %i chega em em: %.1f" % (nome, contaChegada,  
            env.now))
```

SimPy: Dict of Distributions [SimPy Contributors, 2025]

```
(venv) PS C:\Users\user\Desktop\sls\07> py .\ex03-Dicionario-de-distribuicoes.py
3.13.3 (tags/v3.13.3:6280bb5, Apr 8 2025, 14:47:33) [MSC v.1943 64 bit (AMD64)]
chegada: 0.6975252826179686
cantando: 17.36478751005752
aplauso: 11.232275927455648

cliente 1 chega em em: 1.5
cliente 2 chega em em: 2.1
cliente 3 chega em em: 6.0
cliente 4 chega em em: 6.6
cliente 5 chega em em: 7.7
```


SimPy: Call next function [\[SimPy Contributors, 2025\]](#)

```
def geraChegadas(env):
    contaChegada = 0
    while True:
        yield env.timeout(distribuicoes('chegada'))
        contaChegada += 1
        print("%.1f Chegada do cliente %d: " % (env.now,
            contaChegada))
        env.process(atendimentoServidor(env, "cliente %d" %
            contaChegada, servidor))

def atendimentoServidor(env, nome, servidor):
    with servidor.request() as request:
        yield request
        yield env.timeout(distribuicoes('atendimento'))
        print("%.1f Servidor termina atend. do %s. Clientes em
            fila: %i" % (env.now, nome, len(servidor.queue)))

env = simpy.Environment()
servidor = simpy.Resource(env, capacity = 1)
env.process(geraChegadas(env)) # Apenas isso...
env.run(until = 5)
```

SimPy: Call next function [SimPy Contributors, 2025]

```
(venv) PS C:\Users\user\Desktop\s1s\07> py .\ex04-Chama-proxima-funcao-Conta-cliente-em-fila.py
3.13.3 (tags/v3.13.3:6280bb5, Apr 8 2025, 14:47:33) [MSC v.1943 64 bit (AMD64)]
0.5 Chegada do cliente 1:
0.5 Servidor inicia atendimento do cliente 1
1.4 Servidor termina atendimento do cliente 1. Clientes em fila: 0
3.1 Chegada do cliente 2:
3.1 Servidor inicia atendimento do cliente 2
3.3 Chegada do cliente 3:
4.1 Servidor termina atendimento do cliente 2. Clientes em fila: 1
4.1 Servidor inicia atendimento do cliente 3
4.1 Servidor termina atendimento do cliente 3. Clientes em fila: 0
4.3 Chegada do cliente 4:
4.3 Servidor inicia atendimento do cliente 4
4.5 Servidor termina atendimento do cliente 4. Clientes em fila: 0
```

SimPy: Register time and number in queue [SimPy Contributors, 2025]

```
def geraChegadas(env):  
    ...  
  
def atendimentoServidor(env, nome, servidor):  
    chegadaNoServidor = env.now  
    with servidor.request() as request:  
        yield request  
        tempoFila = env.now - chegadaNoServidor  
        print("%.1f Servidor inicia atendimento do %s. Tempo em  
            fila: %.1f" % (env.now, nome, tempoFila))  
  
        yield env.timeout(distribuicoes('atendimento'))  
        print("%.1f Servidor termina atendimento do %s.  
            Clientes em fila: %i" % (env.now, nome,  
                len(servidor.queue)))  
  
env = simpy.Environment()  
servidor = simpy.Resource(env, capacity = 1)  
env.process(geraChegadas(env))  
env.run(until = 15)    # Inicio da Simulacao
```

SimPy: Register time and number in queue [SimPy Contributors, 2025]

```
(venv) PS C:\Users\user\Desktop\sfs\07> py .\ex05-Cria-recurso-Registra-tempo-fila.py  
3.13.3 (tags/v3.13.3:6280bb5, Apr 8 2025, 14:47:33) [MSC v.1943 64 bit (AMD64)]
```

Inicio da Simulacao

0.5 Chegada do cliente 1:

0.5 Servidor inicia atendimento do cliente 1. Tempo em fila: 0.0

1.4 Servidor termina atendimento do cliente 1. Clientes em fila: 0

3.1 Chegada do cliente 2:

3.1 Servidor inicia atendimento do cliente 2. Tempo em fila: 0.0

3.3 Chegada do cliente 3:

4.1 Servidor termina atendimento do cliente 2. Clientes em fila: 1

4.1 Servidor inicia atendimento do cliente 3. Tempo em fila: 0.8

4.1 Servidor termina atendimento do cliente 3. Clientes em fila: 0

4.3 Chegada do cliente 4:

4.3 Servidor inicia atendimento do cliente 4. Tempo em fila: 0.0

4.5 Servidor termina atendimento do cliente 4. Clientes em fila: 0

7.4 Chegada do cliente 5:

7.4 Servidor inicia atendimento do cliente 5. Tempo em fila: 0.0

7.7 Servidor termina atendimento do cliente 5. Clientes em fila: 0

7.8 Chegada do cliente 6:

7.8 Servidor inicia atendimento do cliente 6. Tempo em fila: 0.0

7.8 Servidor termina atendimento do cliente 6. Clientes em fila: 0

SimPy: Seize and release resources [SimPy Contributors, 2025]

```
def chegadaClientes(env, lavadoras, cestos, secadoras):  
    while True:  
        yield env.timeout(distribuicoes('chegadas'))  
        env.process(lavaSeca(env, "Cliente %s" % contaClientes,  
                             lavadoras, cestos, secadoras))  
  
def lavaSeca(env, cliente, lavadoras, cestos, secadoras):  
    req1 = lavadoras.request()  
    yield req1  
    yield env.timeout(distribuicoes('lavar'))  
    req2 = cestos.request() #nao libera lavadora, pega cesto  
    yield req2  
    yield env.timeout(distribuicoes('carregar'))  
    lavadoras.release(req1) # libera a lavadora, nao o cesto  
    req3 = secadoras.request()  
    yield req3 # ocupa a secadora nao libera cesto  
    yield env.timeout(distribuicoes('descarregar'))  
    cestos.release(req2) # libera o cesto mas nao a secadora  
    yield env.timeout(distribuicoes('secar'))  
    secadoras.release(req3) # pode liberar a secadora
```

SimPy: Seize and release resources [SimPy Contributors, 2025]

```
• (venv) PS C:\Users\user\Desktop\sfs\07> py .\ex06-Sequencia-ocupa-e-desocupa-recursos.py
3.13.3 (tags/v3.13.3:6280bb5, Apr  8 2025, 14:47:33) [MSC v.1943 64 bit (AMD64)]

Inicia a Simulacao
4.2 Chegada do cliente 1
4.2 Cliente 1 ocupa lavadora
12.6 Chegada do cliente 2
24.2 Cliente 1 ocupa cesto
26.1 Cliente 1 desocupa lavadora
26.1 Cliente 1 ocupa secadora
26.1 Cliente 2 ocupa lavadora
27.2 Cliente 1 desocupa cesto
37.1 Cliente 1 desocupa secadora
41.0 Chegada do cliente 3
43.3 Chegada do cliente 4
46.1 Cliente 2 ocupa cesto
46.1 Chegada do cliente 5
47.8 Cliente 2 desocupa lavadora
47.8 Cliente 2 ocupa secadora
47.8 Cliente 3 ocupa lavadora
```

SimPy: Select first event of two [SimPy Contributors, 2025]

```
def generateCustomers(env, number, interval, counter):
    for i in range(number):
        c = customer(env, 'Cliente %d' % (i+1), counter, time)
        env.process(c)
        t = random.expovariate(1.0 / interval)
        yield env.timeout(t)

def customer(env, name, counter, time):
    arrive = env.now
    with counter.request() as req:
        pat = random.uniform(1, 3) # min and max patience
        results = yield req | env.timeout(pat) # Wait/abort
        wait = env.now - arrive
        if req in results: # We got to the counter
            print('%4.2f %s Waited %f' % (env.now, name, wait))
            tib = random.expovariate(time)
            yield env.timeout(tib)
            print('%4.2f %s: Fim' % (env.now, name))
        else: # Reneged
            print('%f %s quit after %f' % (env.now, name, wait))
```

SimPy: Select first event of two [SimPy Contributors, 2025]

```
● (venv) PS C:\Users\user\Desktop\s1s\07> py .\ex07-Entidade-eh-funcao-com-recurso-e-evento-condicional.py
3.13.3 (tags/v3.13.3:6280bb5, Apr 8 2025, 14:47:33) [MSC v.1943 64 bit (AMD64)]
Banco Negador

Inicia a Simulacao
0.00 Cliente 01: Cheguei
0.00 Cliente 01: Esperou em fila por: 0.00
3.86 Cliente 01: Fim
5.10 Cliente 02: Cheguei
5.10 Cliente 02: Esperou em fila por: 0.00
6.36 Cliente 03: Cheguei
7.54 Cliente 03: DESISTIU depois de 1.17
17.50 Cliente 04: Cheguei
18.56 Cliente 04: DESISTIU depois de 1.06
18.65 Cliente 02: Fim
20.24 Cliente 05: Cheguei
20.24 Cliente 05: Esperou em fila por: 0.00
20.56 Cliente 05: Fim
```


SimPy: Interruption (Try-Except) [SimPy Contributors, 2025]

```
def palestrante(env):
    try:
        apresentacao = randint(20,35) # Inicio da palestra
        yield env.timeout(apresentacao) # Fim palestra
    except simpy.Interrupt as interrupt:
        print("Moderador finalizou a palestra.")

def moderador(env):
    for i in range(1,11): # 10 palestrantes
        processoPalestrante = env.process(palestrante(env))
        tempo_inicio = env.now
        tempo_limite = env.timeout(30) # Até 30 min
        resultado = yield processoPalestrante | tempo_limite
        print("Tempo limite estourou: {}".format(
            processoPalestrante not in resultado))
        tempo_usado = env.now - tempo_inicio
        print('%d: Palestrante acabou palestra' %tempo_usado)
        if not processoPalestrante.triggered:
            processoPalestrante.interrupt() # Acabou o tempo
```

SimPy: Interruption (Try-Except) [SimPy Contributors, 2025]

```
• (venv) PS C:\Users\user\Desktop\s1s\07> py .\ex08-Try-Except-Interrupt.py
3.13.3 (tags/v3.13.3:6280bb5, Apr 8 2025, 14:47:33) [MSC v.1943 64 bit (AMD64)]
>
Moderador deixou o palestrante 1 começar a palestra
Agora eh: 0
>>
0: Palestrante começou a falar
Palestrante quer falar por 30 minutos
30: Palestra foi finalizada pelo palestrante
Moderador verifica se o palestrante estourou o tempo ou nao
Tempo limite estourou: True
>
Moderador deixou o palestrante 2 começar a palestra
Agora eh: 30
>>
30: Palestrante começou a falar
Palestrante quer falar por 24 minutos
54: Palestra foi finalizada pelo palestrante
Moderador verifica se o palestrante estourou o tempo ou nao
Tempo limite estourou: False
>
Moderador deixou o palestrante 3 começar a palestra
Agora eh: 54
>>
54: Palestrante começou a falar
Palestrante quer falar por 32 minutos
Moderador verifica se o palestrante estourou o tempo ou nao
Tempo limite estourou: True
Time now: 84
processo_palestrante.is_alive: True
84: Moderador interrompeu e finalizou a palestra
Moderador finalizou a palestra.
```

SimPy: Processes (with limited Resource) [SimPy Contributors, 2025]

```
def participante(env, nome, buffet, conhecimento=0, fome=0):
    while True:
        for i in range(3): # falas por sessao
            conhecimento += randint(0, 3) / (1 + fome)
            fome += randint(1, 4)
            yield env.timeout(30) # duracao da fala (acabou)
        start = env.now # Vai ao Buffet
        with buffet.request() as req:
            yield req | env.timeout(15-3) #Int - t. comendo
            time_left = 15 - (env.now - start)
            if req.triggered:
                comida = min(randint(3,12), time_left)
                yield env.timeout(3) #Tempo comendo
                fome -= min(comida,fome)
                time_left -= 3 # Tempo comendo
            else:
                fome += 1 #Penalidade por caminhar e nao comer
            yield env.timeout(time_left)
```

SimPy: Processes (with limited Resource) [SimPy Contributors, 2025]

```
env = simpy.Environment()
buffet = simpy.Resource(env, capacity=1) # Cap. buffet

for i in range(5):
    env.process(participante(env, 1+i, buffet))
env.run(until = 220)
```

SimPy: Processes (with limited Resource) [SimPy Contributors, 2025]

```
● (venv) PS C:\Users\user\Desktop\sfs\07> py .\ex09-Recurso-limitado-Criacao-de-processos.py
3.13.3 (tags/v3.13.3:6280bb5, Apr 8 2025, 14:47:33) [MSC v.1943 64 bit (AMD64)]
90: Participante 1 terminou sua fala com conhecimento 4.20 e fome 7.00
90: Participante 2 terminou sua fala com conhecimento 1.83 e fome 8.00
90: Participante 3 terminou sua fala com conhecimento 3.52 e fome 9.00
90: Participante 4 terminou sua fala com conhecimento 2.50 e fome 3.00
90: Participante 5 terminou sua fala com conhecimento 0.17 e fome 9.00
93: Participante 1 terminou de comer com tempo 10.00 e fome 0.00
96: Participante 2 terminou de comer com tempo 7.00 e fome 1.00
99: Participante 3 terminou de comer com tempo 6.00 e fome 3.00
102: Participante 4 terminou de comer com tempo 6.00 e fome 0.00
102: > Participante 5 nao chegou no buffet, sua fome agora esta em 10.00

195: Participante 1 terminou sua fala com conhecimento 7.88 e fome 10.00
195: Participante 2 terminou sua fala com conhecimento 3.58 e fome 9.00
195: Participante 3 terminou sua fala com conhecimento 4.42 e fome 8.00
195: Participante 4 terminou sua fala com conhecimento 4.50 e fome 5.00
195: Participante 5 terminou sua fala com conhecimento 0.68 e fome 14.00
198: Participante 1 terminou de comer com tempo 12.00 e fome 0.00
201: Participante 2 terminou de comer com tempo 10.00 e fome 0.00
204: Participante 3 terminou de comer com tempo 3.00 e fome 5.00
207: Participante 4 terminou de comer com tempo 3.00 e fome 2.00
207: > Participante 5 nao chegou no buffet, sua fome agora esta em 15.00
```

```
def clock(env, name, tick):  
    while True:  
        print(name, env.now)  
        yield env.timeout(tick)  
  
env = simpy.Environment()  
  
env.process(clock(env, 'rapido', 0.5))  
env.process(clock(env, 'devagar', 1))  
  
env.run(until = 3.1)
```

SimPy: Parallel Processes [SimPy Contributors, 2025]

```
• (venv) PS C:\Users\user\Desktop\s1s\07> py .\ex10-Processos-diferentes.py
3.13.3 (tags/v3.13.3:6280bb5, Apr 8 2025, 14:47:33) [MSC v.1943 64 bit (AMD64)]
rapido 0
devagar 0
rapido 0.5
devagar 1
rapido 1.0
rapido 1.5
devagar 2
rapido 2.0
rapido 2.5
devagar 3
rapido 3.0
```

SimPy: main() at the end [SimPy Contributors, 2025]

```
def main():
    env = simpy.Environment()
    env.process(traffic_light(env))
    print("Simulacao do Semaforo")
    env.run(until = 120)
    print("Simulacao Completa")

def traffic_light(env):
    while True:
        print("%4.2f Luz VERDE" %(env.now))
        yield env.timeout(30)
        print("%4.2f Luz AMARELA" %(env.now))
        yield env.timeout(5)
        print("%4.2f Luz VERMELHA" %(env.now))
        yield env.timeout(20)

if __name__ == '__main__':
    main()
```


SimPy: main() at the end [SimPy Contributors, 2025]

```
● (venv) PS C:\Users\user\Desktop\s1s\07> py .\ex11-Funcao-main-ao-final.py
Simulacao do Semaforo
0.00 Luz VERDE
30.00 Luz AMARELA
35.00 Luz VERMELHA
55.00 Luz VERDE
85.00 Luz AMARELA
90.00 Luz VERMELHA
110.00 Luz VERDE
Simulacao Completa
```

Outline of this lecture

Simpy Models

SimPy Exercise

Simpy Models

SimPy Exercise

SimPy Exercise: Improve your project model

Exercise (10 pts): The group should **implement** all previous examples **adding** print function within the code to show the simulation progress.

Following, the group should **improve** the simulation project model using the SimPy features listed in this class and, if necessary, redesign the ACD model.

References



Medina, A. C. (2025).

SimPy: Simulação em Python — Um guia prático.

Leanpub.



Python Contributors (2025).

Glossary: “generator” — python 3.13.7 documentation.

<https://docs.python.org/3/glossary.html#term-generator>.



SimPy Contributors (2025).

Simpy – discrete-event simulation for python.

<https://simpy.readthedocs.io/en/latest/index.html>.